UNIVERSIDAD DE SAN BUENAVENTURA

Optativa II · Ingeniería de Software

MARKETPLACE

Manual Técnico

Guía para desarrolladores y mantenimiento

Basado en IEEE 1063 / ISO/IEC/IEEE 26514 · Código MKT-MTE-001 · Versión 2.0

Integrante	Rol Scrum
Sarah Isabella Peña	Product Owner
Juan Carlos Diaz	Scrum Master
Sebastian Grande Moreno	Developer

Docente: Ing. Jairo Armando Salcedo Aranda

Bogotá D.C., 3 de noviembre de 2026

Control de versiones

Versión	Fecha	Autor	Descripción
1.0	02/11/2026	Equipo Marketplace	Versión inicial: autenticación, catálogo, carrito, pedidos y chatbot.
2.0	03/11/2026	Equipo Marketplace	Perfiles Comprador y Administrador, favoritos, historial de pedidos, panel y gestión de catálogo.

Tabla de contenido

Control de versiones....2
Tabla de contenido....2
1. Introducción....3
2. Stack tecnológico....3
3. Estructura del repositorio....3
4. Componentes principales....3
4.1 MarketViewModel....3
4.2 Lógica de dominio....4
4.3 Pantallas....4
5. Persistencia....4
5.1 Firestore....4
5.2 Catálogo local....5
6. Cómo extender....5
7. Convenciones....5
8. Comandos útiles....5

1. Introducción

Este manual describe la estructura del código, las clases principales, la persistencia y cómo extender Marketplace. Complementa el documento de Arquitectura (MKT-SAD-001).

2. Stack tecnológico

Tecnología	Versión	Uso
Kotlin	1.9.24	Lenguaje
Jetpack Compose BOM	2024.06.00	UI declarativa (Material 3)
Android Gradle Plugin	8.5.2	Compilación
Gradle	8.7	Automatización
Lifecycle ViewModel	2.8.3	Estado
Firebase BoM	33.1.2	Auth + Firestore
JUnit	4.13.2	Pruebas unitarias
Azure Pipelines	—	CI/CD

3. Estructura del repositorio

```
marketplace/
├── app/                  # módulo Android (código y pruebas)
├── firebase/             # firestore.rules (sin cambios)
├── docs/                 # requisitos, backlog, arquitectura, Firebase
├── Documentacion/        # documentos IEEE, matrices, manuales, modelos
├── azure-pipelines.yml
└── README.md
```

4. Componentes principales

4.1 MarketViewModel

Grupo	Estado / funciones
Sesión	user, isAdmin, login(), register(), resetPassword(), logout()
Catálogo	catalog (todos), products (activos), productById(), query, category
Carrito	cart, cartLines, cartCount, addToCart(), decrement(), remove(), checkout()
Perfil	profile, favorites, orders, updateProfile(), toggleFavorite(), loadOrders(), sendPasswordChange()
Administración	saveProduct(), deleteProduct(), setProductActive(), restock(), resetCatalog() — todas validan requireAdmin()
Chat	chat, typing, send()

4.2 Lógica de dominio

Objeto	Archivo	Responsabilidad
AdminConfig / Role	data/Roles.kt	Asignar el rol por correo.
CatalogStore	data/CatalogStore.kt	Cargar/guardar/restaurar el catálogo (JSON en SharedPreferences).
CatalogSearch, CartOps, Pricing	logic/Logic.kt	Búsqueda, carrito, precios y envío.
Validators, AuthErrors	logic/Logic.kt	Validar registro/login y traducir errores de Firebase.
MarketBot	logic/MarketBot.kt	Asistente basado en reglas.
ProductValidator, ProductDraft	logic/AdminLogic.kt	Validar y convertir el formulario de producto.
CatalogOps	logic/AdminLogic.kt	nextId, upsert, delete, setActive, adjustStock, applyOrder, visible.
Inventory	logic/AdminLogic.kt	Indicadores del panel (LOW_STOCK = 10).
ProfileValidator, Initials, Dates	logic/ProfileLogic.kt	Validar perfil y formatear.
OrderParser, OrderSummary	logic/ProfileLogic.kt	Convertir documentos de orders en objetos.

4.3 Pantallas

Pantalla	Rol	Descripción
AuthScreen	Ambos	Login, registro y recuperación.
CatalogScreen	Comprador (y admin en solo lectura)	Búsqueda, categorías, favoritos, detalle.
CartScreen	Comprador	Carrito, resumen y compra.
ChatScreen	Comprador	Asistente Mari.
ProfileScreen	Ambos	Datos, edición, favoritos, pedidos, permisos, seguridad.
AdminDashboardScreen	Administrador	Indicadores, valor por categoría, stock bajo.
AdminProductsScreen	Administrador	Listado, búsqueda, CRUD, interruptor, stock, restaurar.

5. Persistencia

5.1 Firestore

```
users/{uid}  { name, email, role, phone, address, city, favorites: [1, 5], createdAt,
updatedAt }
carts/{uid}  { items: { "3": 2, "5": 1 }, updatedAt }
orders/{id}  { uid, email, items: [{productId, name, quantity, price}], subtotal,
shipping,
               total, shippingAddress, status, createdAt }
```

Todas las escrituras de perfil usan `SetOptions.merge()` para no borrar campos existentes. Los nuevos campos (`role`, `phone`, `address`, `city`, `favorites`) caben en las reglas actuales porque el usuario escribe solo su propio documento.

5.2 Catálogo local

SharedPreferences `marketplace_catalog`, clave `products_v1`: arreglo JSON de productos. Si no existe o está corrupto, se usa `ProductRepository.products`.

6. Cómo extender

Agregar un administrador

Añadir el correo en `AdminConfig.ADMIN_EMAILS` y recompilar.

Agregar una categoría

Crear la constante en `ProductRepository`, incluirla en `categories` y agregar palabras clave en `MarketBot.CATEGORY_WORDS`.

Agregar un indicador al panel

Agregar el campo en `InventoryStats`, calcularlo en `Inventory.stats`, probarlo en `AdminLogicTest` y mostrarlo con `MiniStat` en `AdminDashboardScreen`.

Migrar el catálogo a Firestore (futuro)

Reemplazar `CatalogStore` por un repositorio sobre la colección `products` y añadir una regla de escritura solo para administradores (requiere autorizar cambios en Firebase).

7. Convenciones

- Ramas: `main` (estable, con PR), `develop` (integración), `feature/<nombre>`.
- Lógica nueva = objeto puro en `logic/` + prueba en `app/src/test`.
- Textos de interfaz en español; mensajes de error claros y accionables.
- `testTag` en los componentes interactivos para pruebas de UI.

8. Comandos útiles

```
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
./gradlew clean
```